Universidad de Murcia

Facultad de Informática

Máster Universitario en Inteligencia Artificial

Extensiones de Machine Learning

Autores:
Juan Eleazar Escudero Candel
Raúl Rodríguez Alburquerque
Jesús Sánchez Ochoa

Correos:
juaneleazar.escudero@um.es
raul.rodrigueza@um.es
jesus.sanchezo@um.es

15 de marzo de 2026

Resumen

En este documento vamos a hablar sobre sobre nuestro estudio del aprendizaje por refuerzo, en el que hemos podido ver como agentes inteligentes toman decisiones en un entorno para maxímizar la obtención de recompensas. En estos algoritmos hemos podido apreciar el dilema de la exploración vs. explotación, una de las partes más importantes de esta doctrina, en la cual hemos visto como buscar un equilibrio entre explorar opciones y explotar las que más recompensas nos generan.

En el primer capítulo vamos a hablar sobre el problema del bandido de k-brazos, que es una simplificación del aprendizaje por refuerzo en el cual utilizamos entornos estacionarios. Aquí hemos podido probar los algoritmos ϵ-Greedy, UCB1 y Softmax para las distribuciones normal, bernoulli y binomial y hemos comprobado la importancia de encontrar un equilibrio entre exploración y explotación, exceptuando para la distribución binomial en la que era más importante la explotación.

En el segundo capítulo vamos a hablar sobre el aprendizaje en entornos complejos, en los cuales tenemos entornos con diferentes estados, acciones para transicionar entre ellos y recompensas asociadas a cada una. Aquí hemos empezado estudiando los metodos tabulares, compuestos de Monte Carlo, SARSA y Q-Learning. Para acabar, hemos estudiado los algoritmos de control con aproximaciones, compuesto por SARSA semi-gradiente y Deep Q-Learning. En este capitulo hemos mantenido el dilema de exploración vs explotación, y hemos podido ver el funcionamiento de algoritmos más complejos necesarios para escenarios no estacionarios, más similares a lo que podemos encontrarnos en la vida real.

Índice general

1. Problema del Bandido de k-brazos **2**
1.1. Introducción . . . 2
1.2. Desarrollo . . . 2
1.2.1. Marco Teórico: Aprendizaje por Refuerzo Simplificado . . . 2
1.2.2. Definición Formal del Problema . . . 2
1.2.3. El Dilema de Exploración vs. Explotación . . . 3
1.2.4. Trabajos Relacionados y Enfoques en la Literatura . . . 3
1.3. Algoritmos . . . 3
1.3.1. Definición de la Estructura Base . . . 3
1.3.2. Algoritmo ϵ-Greedy . . . 4
1.3.3. Algoritmo UCB1 (Upper Confidence Bound) . . . 4
1.3.4. Algoritmo Softmax (Exploración de Boltzmann) . . . 4
1.4. Evaluación/Experimentos . . . 5
1.4.1. Análisis de Resultados . . . 5
1.4.2. Comparación entre Distribuciones . . . 7
1.4.3. Comparación entre Familias de algoritmos . . . 7
1.5. Conclusiones . . . 8

2. Aprendizaje en Entornos Complejos **9**
2.1. Introducción . . . 9
2.2. Desarrollo . . . 9
2.2.1. Marco Teórico: Aprendizaje en entornos complejos . . . 9
2.2.2. Definición Formal del Problema . . . 10
2.2.3. Dilema Exploración vs. Explotación . . . 10
2.2.4. Trabajos Relacionados y Enfoques en la Literatura . . . 10
2.3. Algoritmos . . . 11
2.3.1. Definición de la Estructura Base . . . 11
2.3.2. Algoritmo Monte Carlo . . . 11
2.3.3. Algoritmo SARSA . . . 11
2.3.4. Algoritmo Q-Learning . . . 11
2.3.5. Algoritmo SARSA Semi-Gradiente . . . 12
2.3.6. Algoritmo Deep Q-Learning . . . 12
2.4. Experimentos . . . 13
2.4.1. Entorno de Blackjack . . . 13
2.4.2. Entorno de Mountain Car . . . 17
2.5. Conclusiones . . . 20

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

El problema del Bandido de k-brazos es el mejor ejemplo del dilema de **Exploración-Explotación** en el contexto del aprendizaje por refuerzo. Suponiendo que tenemos un bandido que puede tomar k acciones, el objetivo es encontrar un algoritmo que consiga un equilibrio entre explorar todas las diferentes opciones y explotar la acción que mejores resultados nos ha generado hasta el momento.

Para conseguir esto, vamos a probar algoritmos de diferentes familias, que definirán la forma de elegir las diferentes acciones, para poder comparar los resultados obtenidos por cada familia. En nuestro caso concreto, vamos a comparar las familias de algoritmos Voraz (Greedy), Límite Superior de Confianza (UCB) y Ascenso del Gradiente (Gradient Ascent). Además, también utilizaremos diferentes distribuciones, que definirán los valores de las recompensas de cada acción. De esta forma, podremos comparar el funcionamiento de varios algoritmos en diferentes entornos.

El resto de la memoria está separada en un apartado para explicar en más detalle el problema a tratar, un apartado de algoritmos en el que detallaremos más sobre los algoritmos utilizados, un apartado de evaluación/experimentos para explicar los resultados obtenidos y los experimentos realizados para obtenerlos, y por último, un apartado de conclusiones en el que resumiremos los principales resultados que hemos obtenido.

1.2. Desarrollo

En este apartado se establecen las bases teóricas necesarias para comprender la problemática del bandido de k-brazos, situándolo dentro del campo del Aprendizaje por Refuerzo (RL) y definiendo formalmente el objetivo de optimización.

1.2.1. Marco Teórico: Aprendizaje por Refuerzo Simplificado

El problema del bandido multibrazo puede considerarse un caso especial y simplificado del Aprendizaje por Refuerzo. En el RL estándar, un agente interactúa con un entorno a través de una secuencia de estados, acciones y recompensas. Sin embargo, el problema del bandido se caracteriza por ser **sin estado** (o de un solo estado).

Esto implica que la acción tomada por el agente en el instante t no influye en el estado del entorno en el instante $t+1$, sino únicamente en la recompensa inmediata recibida. El objetivo del agente no es cambiar el entorno, sino descubrir cuál de las opciones disponibles ofrece la mayor ganancia esperada.

1.2.2. Definición Formal del Problema

Consideramos un problema de decisión secuencial en el que un agente debe elegir entre k opciones o "brazos" diferentes. En cada paso de tiempo discreto $t = 1, 2, \ldots, T$, el agente selecciona una acción A_t del conjunto de acciones posibles $\mathcal{A} = \{1, \ldots, k\}$ y recibe una recompensa numérica $R_t \in \mathbb{R}$.

Cada acción a tiene asociada una distribución de probabilidad desconocida para el agente, con una esperanza o valor esperado denotado como $q_*(a)$:

$$q_*(a) \doteq \mathbb{E}[R_{t+1} | A_t = a] \tag{1.1}$$

El objetivo del agente es maximizar la recompensa

total acumulada a lo largo de un horizonte de tiempo T. Si el agente conociera los valores reales $q_*(a)$, la estrategia óptima sería trivial: elegir siempre la acción con el valor esperado máximo, denotada como a^*:

$$a^* = \operatorname*{argmax}_{a \in \mathcal{A}} q_*(a) \tag{1.2}$$

Sin embargo, dado que el agente desconoce estas distribuciones, debe estimarlas basándose en las observaciones.

Denotamos $Q_t(a)$ como la estimación del valor de la acción a en el instante t. Se espera que, con suficiente exploración, $Q_t(a) \to q_*(a)$.

1.2.3. El Dilema de Exploración vs. Explotación

La dificultad central del problema radica en el conflicto entre dos comportamientos opuestos:

- **Explotación:** Seleccionar la acción que, según el conocimiento actual, produce la mayor recompensa ($A_t = \operatorname{argmax} Q_t(a)$). Esto maximiza la ganancia a corto plazo.
- **Exploración:** Seleccionar una acción distinta a la que se considera óptima actualmente para recopilar más información sobre su valor real. Esto puede implicar una pérdida a corto plazo, pero es esencial para descubrir acciones superiores a largo plazo.

Los algoritmos implementados en este trabajo (ϵ-greedy, UCB1 y Softmax) representan diferentes enfoques matemáticos para gestionar este compromiso.

1.2.4. Trabajos Relacionados y Enfoques en la Literatura

El problema del bandido de k-brazos fue formulado originalmente por Herbert Robbins (1952) y ha sido extensamente estudiado debido a sus aplicaciones en ensayos clínicos, optimización de publicidad online y enrutamiento de redes.

En la literatura existen diversas familias de soluciones:

- **Enfoques Semi-Uniformes:** Como el método ϵ-greedy, que inyecta ruido aleatorio en la toma de decisiones. Son simples computacionalmente pero efectivos en entornos estacionarios.
- **Soluciones de Intervalo de Confianza:** Como la familia UCB (Upper Confidence Bound) introducida por Auer et al. [6]. Estos métodos construyen límites superiores estadísticos para el valor de cada brazo, eligiendo el que tiene el mayor potencial bajo el principio de "optimismo ante la incertidumbre".
- **Métodos Probabilísticos (Boltzmann):** Como el algoritmo Softmax. A diferencia de los métodos uniformes, estos asignan probabilidades de selección graduadas: las acciones con mayores estimaciones $Q_t(a)$ tienen mayor probabilidad de ser elegidas, mejorando la eficiencia de la exploración.
- **Algoritmos de Gradiente:** A diferencia de Softmax (que usa estimaciones de valor), estos métodos aprenden "preferencias numéricas" para cada acción utilizando ascenso de gradiente estocástico sobre la recompensa media recibida.
- **Muestreo de Thompson:** Un enfoque Bayesiano que mantiene una distribución de probabilidad posterior para la media de cada brazo y selecciona acciones muestreando de estas distribuciones.

Este trabajo se centra en una comparativa empírica entre las tres primeras estrategias, de las cuales utilizaremos los modelos ϵ-greedy, UCB1 y Softmax, bajo distribuciones de recompensa con características distintas, permitiendo observar cómo la varianza del entorno afecta la convergencia de cada estimador.

1.3. Algoritmos

En esta sección se van a discutir los diferentes algoritmos que se han implementado. Para esta práctica, se han desarrollado los algoritmos ϵ-greedy, UCB1 y Softmax.

1.3.1. Definición de la Estructura Base

Para garantizar una comparación justa y modular entre las diferentes estrategias, se ha implementado primero una clase abstracta base (`Algorithm`) que

define la estructura común para todos los agentes del problema del bandido k-brazos.

Esta estructura se encarga de almacenar dos vectores fundamentales para el aprendizaje:

- **Conteo de selecciones ($N_t(a)$)**: El número de veces que el brazo a ha sido seleccionado hasta el instante t.
- **Estimación de valor ($Q_t(a)$)**: La recompensa promedio estimada para el brazo a.

La actualización de las estimaciones de valor se realiza de manera incremental para optimizar la memoria y el cómputo. En lugar de almacenar todo el historial de recompensas, se utiliza la regla de actualización de la media incremental [4]:

$$Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n) \tag{1.3}$$

Donde Q_n es la estimación actual, R_n es la recompensa obtenida en el paso n-ésimo para ese brazo, y Q_{n+1} es la nueva estimación. Esta lógica se encuentra encapsulada en el método `update` de la clase base.

1.3.2. Algoritmo ϵ-Greedy

El algoritmo ϵ-greedy es una de las estrategias fundamentales en el aprendizaje por refuerzo que busca equilibrar el el *trade-off* exploración-explotación.

La estrategia se basa en un parámetro $\epsilon \in [0, 1]$, que define la probabilidad de realizar una acción exploratoria. En cada paso de tiempo t:

1. Con probabilidad $1 - \epsilon$, el agente **explota** su conocimiento actual, seleccionando el brazo con la mayor recompensa estimada (acción codiciosa o *greedy*): $A_t = \text{argmax}_a Q_t(a)$.
2. Con probabilidad ϵ, el agente **explora**, seleccionando un brazo cualquiera de manera uniformemente aleatoria, independientemente de sus valores estimados.

Esta simple regla asegura que, si $N_t(a) \to \infty$, las estimaciones $Q_t(a)$ convergerán a los valores reales $q_*(a)$ [5].

1.3.3. Algoritmo UCB1 (Upper Confidence Bound)

A diferencia del método ϵ-greedy, que se basa en una exploración aleatoria uniforme, el algoritmo UCB1 adopta un enfoque determinista basado en el principio de *optimismo ante la incertidumbre*.

La idea central es seleccionar acciones basándose no solo en su estimación de valor actual ($Q_t(a)$), sino también en la incertidumbre asociada a dicha estimación. Para cada brazo a y en cada paso de tiempo t, se calcula un índice de confianza superior, dado por la fórmula [6]:

$$A_t = \text{argmax}_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right] \tag{1.4}$$

Esta expresión se compone de dos términos contrapuestos:

- **Término de Explotación ($Q_t(a)$):** Fomenta la selección de brazos que han demostrado tener una recompensa promedio alta hasta el momento.
- **Término de Exploración ($c\sqrt{\frac{\ln t}{N_t(a)}}$):** Representa la incertidumbre o el intervalo de confianza. Si un brazo ha sido seleccionado pocas veces ($N_t(a)$ es pequeño), este término será grande, forzando al algoritmo a probarlo. El parámetro c regula el grado de exploración.

A medida que avanza el tiempo (t crece), el término $\ln t$ aumenta lentamente, asegurando que la exploración nunca se detenga por completo, pero se reduzca progresivamente a medida que se acumula evidencia.

1.3.4. Algoritmo Softmax (Exploración de Boltzmann)

El algoritmo Softmax aborda una de las principales limitaciones de ϵ-greedy: la falta de discriminación durante la exploración. Mientras que ϵ-greedy elige cualquier brazo no óptimo con la misma probabilidad, Softmax asigna probabilidades de selección proporcionales a la recompensa estimada de cada brazo [7].

La selección de acciones se basa en el axioma de elección de Luce [8], modelado comúnmente mediante

una distribución de Boltzmann (o Gibbs). La probabilidad $\pi_t(a)$ de seleccionar el brazo a en el instante t se define como:

$$\pi_t(a) = \frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^{k} e^{Q_t(b)/\tau}} \tag{1.5}$$

Donde τ (tau) es el parámetro de **temperatura**. Según la literatura clásica de aprendizaje por refuerzo [4]:

- **Alta temperatura ($\tau \to \infty$):** Las probabilidades tienden a igualarse, resultando en una exploración casi uniforme.
- **Baja temperatura ($\tau \to 0$):** Las probabilidades se concentran en el brazo con mayor $Q_t(a)$, tendiendo a una explotación pura.

1.4. Evaluación/Experimentos

Para garantizar la robustez de los resultados, se diseñó un entorno de pruebas estandarizado con las siguientes características:

- **Entorno:** Bandido de $k = 10$ brazos, donde las probabilidades de éxito p de cada brazo se generaron aleatoriamente siguiendo una distribución que definirá el rango de posibles valores.
- **Distribuciones de Recompensa:** Se evaluaron tres escenarios distintos para probar la adaptabilidad de los agentes:
  - *Normal (Gaussiana):* Recompensas continuas, modelando ruido natural.
  - *Binomial:* Recompensas discretas basadas en n intentos (con n=10).
  - *Bernoulli:* Recompensas binarias (0 o 1), representando el escenario más ruidoso y desafiante.
- **Algoritmos Comparados:**
  - **ϵ-Greedy:** Con valores de $\epsilon \in \{0, 0.1, 0.01\}$.
  - **Softmax (Boltzmann):** Con temperaturas $\tau \in \{0.1, 1, 5\}$.
  - **UCB1:** Con constantes de exploración $c \in \{0.1, 1, 5\}$.
- **Métricas:** Se utilizaron la *Recompensa Promedio* para medir el rendimiento instantáneo y el *Porcentaje de Selección Óptima* y *Ganancias vs Perdidas* para evaluar la precisión de seleccionar el brazo óptimo.

1.4.1. Análisis de Resultados

A continuación, se presenta el análisis de resultados desglosado por familia de algoritmos y su comportamiento frente a las distintas distribuciones.

1.4.1.1. Familia Epsilon-Greedy

El análisis revela una clara tensión entre la velocidad de aprendizaje y la eficiencia final (Gráfica para distribución normal en notebook).

- $\epsilon = 0$ **(Greedy puro):** Sufre de *convergencia prematura*. En todas las distribuciones, el agente queda atrapado en óptimos locales, ignorando el brazo óptimo al carecer de exploración y mostrando los peores resultados.
- $\epsilon = 0.1$ **(Exploración Alta):** Destaca por su velocidad inicial, ideal para horizontes temporales cortos. Sin embargo, su techo de rendimiento es limitado debido al hecho de tener que realizar exploración aleatoria (10 % de las veces).
- $\epsilon = 0.01$ **(Exploración Conservadora):** Demostró ser la estrategia más eficiente a largo plazo (horizontes > 2000 pasos). Aunque su arranque es lento, una vez identifica el brazo óptimo, lo explota el 99 % del tiempo, superando en recompensa acumulada a la configuración de $\epsilon = 0.1$.

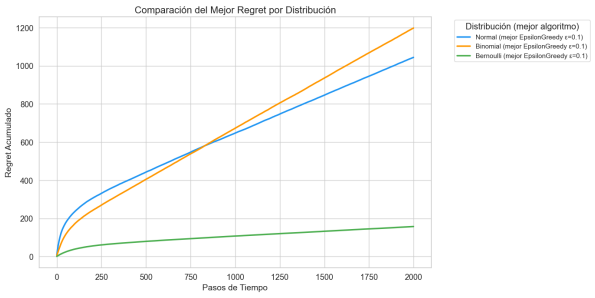


Figura 1.1: Regret del algoritmo $e - Greedy$ por pasos de tiempo.

La figura 1.1 compara el mejor regret acumulado obtenido por ϵ-Greedy ($\epsilon = 0.1$ en los tres casos) por distribución. La distribución Bernoulli destaca nuevamente como la más favorable, con un crecimiento lento y tendencia a estabilizarse. Las distribuciones Normal y Binomial presentan un crecimiento prácticamente lineal y sostenido, sin señales de conver-

gencia, alcanzando valores en torno a 1000–1200 al final del horizonte. A diferencia de UCB1, donde Normal y Binomial convergían rápidamente, aquí ϵ-Greedy no logra estabilizar el regret en estos entornos, lo que refleja la limitación de mantener una tasa de exploración constante una vez identificado el brazo óptimo.

1.4.1.2. Familia Softmax

Softmax mostró una alta sensibilidad al parámetro de temperatura τ, que regula la probabilidad de elegir brazos según la magnitud de sus recompensas estimadas (Gráfica para distribución normal en notebook).

- $\tau = 0.1$ **(Baja Temperatura):** Comportamiento casi determinista. Salvo en entornos muy sencillos, tiende a atascarse en brazos subóptimos de forma similar al enfoque Greedy.
- $\tau = 1$ **(Balanceado):** Ofreció el mejor equilibrio en distribuciones Normales y Binomiales, logrando identificar brazos de alta recompensa sin sacrificar excesiva exploración.
- $\tau = 5$ **(Alta Temperatura):** Resultó en una exploración excesiva e ineficiente. El agente actúa de forma demasiado aleatoria, impidiendo la estabilización en el brazo óptimo incluso después de muchos pasos.

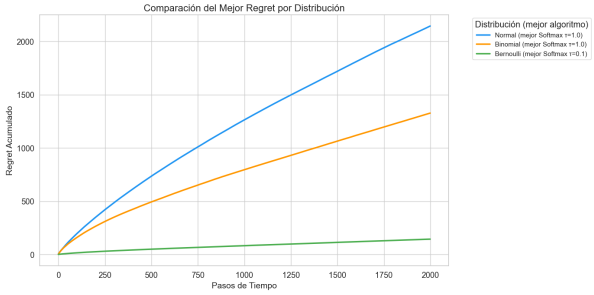


Figura 1.2: Regret de Softmax por pasos de tiempo

La figura 1.2 compara el mejor regret acumulado obtenido por Softmax en cada distribución. En este caso, la distribución Bernoulli ($\tau = 0.1$) es la que mejor comportamiento presenta, con un crecimiento muy lento que se aproxima a la estabilización al final del horizonte. Las distribuciones Normal y Binomial ($\tau = 1.0$ en ambos casos) muestran un crecimiento continuo y sostenido sin señales de convergencia, alcanzando valores muy superiores. Destaca

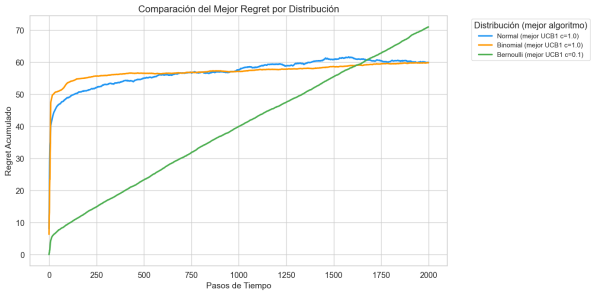


Figura 1.3: Regret de UCB1 por pasos de tiempo.

especialmente la distribución Normal, cuyo regret acumulado casi duplica al de la Binomial, lo que sugiere que Softmax encuentra mayores dificultades para distinguir entre brazos en entornos con recompensas continuas y alta varianza.

1.4.1.3. Familia UCB1

UCB1 gestiona la incertidumbre mediante intervalos de confianza, ofreciendo resultados robustos pero dependientes de la escala de exploración c (Gráfica para distribución normal en notebook).

- $c = 0.1$ **y** $c = 1$**:** Mostraron una convergencia rápida y recompensas altas en las distribuciones Normal y Binomial. Sin embargo, en la distribución de Bernoulli (la más ruidosa), se observó que priorizar la explotación ($c = 0.1$) ofreció una convergencia más rápida que el enfoque balanceado ($c = 1$), sugiriendo que en entornos binarios la sobre-exploración es costosa.
- $c = 5$**:** Generó una sobre-exploración contraproducente. El algoritmo sobreestima la incertidumbre de los brazos no probados, perdiendo demasiado tiempo verificando opciones que ya deberían haber sido descartadas.

La figura 1.3 compara el mejor regret acumulado obtenido por UCB1 en cada distribución. Las curvas de Normal ($c = 1.0$) y Binomial ($c = 1.0$) presentan un comportamiento similar: el regret crece rápidamente en los primeros pasos y se estabiliza en torno a 60, lo que indica que el algoritmo identifica el brazo óptimo con relativa rapidez. La distribución Bernoulli ($c = 0.1$), en cambio, muestra un crecimiento lineal sostenido a lo largo de todo el horizonte, sin señales de estabilización, lo que sugiere que UCB1

encuentra mayores dificultades para converger en este tipo de recompensas.

1.4.2. Comparación entre Distribuciones

Se observó que la naturaleza de la recompensa afecta drásticamente al aprendizaje:

- **Normal:** Curvas de aprendizaje suaves; fácil diferenciación de medias.
- **Bernoulli:** El entorno más desafiante. La alta varianza de una señal binaria (0/1) requiere periodos de exploración más largos para evitar descartar falsamente al brazo óptimo tras una mala racha inicial.
- **Binomial:** Representa un escenario intermedio. Aunque permite recompensas más altas que la Bernoulli, la naturaleza discreta de los resultados (conteo de éxitos en n=10 intentos) introduce una variabilidad inicial que hace que la convergencia sea ligeramente más inestable que en la distribución Normal, aunque finalmente alcanza rendimientos elevados.

1.4.3. Comparación entre Familias de algoritmos

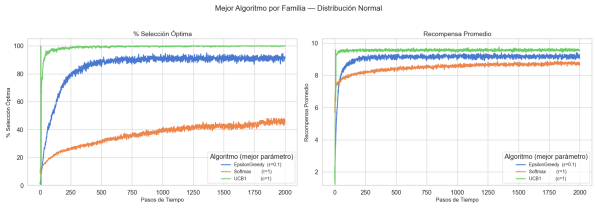


Figura 1.4: Comparativa de mejores algoritmos para la distribución normal.

En el entorno modelado por una distribución Normal (figura 1.4), observamos una correlación directa y positiva entre ambas métricas. El algoritmo que logra el mayor porcentaje de selección del brazo óptimo en la fase final es, consecuentemente, el que obtiene la mayor recompensa promedio. En un espacio de recompensas continuas, identificar y explotar la media más alta (μ) se traduce matemáticamente en el máximo retorno posible, por lo que ambas métricas están perfectamente alineadas.

El comportamiento observado en la distribución Binomial (figura 1.5) es análogo al de la Normal. Al

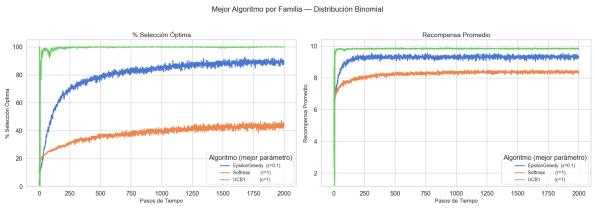


Figura 1.5: Comparativa de mejores algoritmos para la distribución binomial.

tratarse de una suma de éxitos (n ensayos con probabilidad p), el gradiente de recompensas esperadas entre los distintos brazos permite que la convergencia hacia el brazo óptimo se refleje de manera proporcional en la recompensa promedio. Nuevamente, el algoritmo que más selecciona la acción óptima es el ganador indiscutible en rendimiento acumulado.

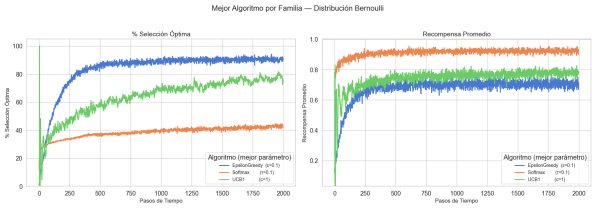


Figura 1.6: Comparativa de mejores algoritmos para la distribución Bernoulli.

Sin embargo, en la distribución de Bernoulli, entorno estrictamente binario de éxito/fracaso (figura 1.6), presenciamos una divergencia fundamental. ϵ-greedy logra el mayor porcentaje de selección del brazo óptimo, pero paradójicamente obtiene una menor recompensa promedio. Esto ocurre porque su estrategia de exploración es ciega: cuando decide explorar, elige aleatoriamente entre todos los brazos restantes con la misma probabilidad, incluyendo el peor de todos. En un entorno binario, cada error penaliza gravemente al ser un 0. Softmax, por el contrario, selecciona el brazo estrictamente óptimo menos veces, pero concentra su probabilidad de exploración en los brazos subóptimos que tienen un mayor valor esperado. Es decir, cuando Softmax "se equivoca" y no elige el brazo número 1, es muy probable que elija el brazo número 2, el cual sigue teniendo una alta probabilidad de devolver un 1 (éxito). Esto demuestra que la métrica de porcentaje de seleccion óptima puede resultar enga-

ñosa en entornos de recompensa discreta.

1.5. Conclusiones

El estudio realizado sobre el problema del *Multi-Armed Bandit* ha permitido corroborar empíricamente los dilemas fundamentales del aprendizaje por refuerzo. A continuación se resumen las principales conclusiones:

- **Resumen de resultados principales.** La comparación empírica en un bandido de $k = 10$ brazos confirma que el rendimiento no depende solo del algoritmo, sino del *régimen de exploración* que impone su hiperparámetro y del horizonte temporal. En general, el mejor rendimiento se ha obtenido al conseguir un equilibrio con la exploración y explotación, para evitar tanto una baja exploración que haga que el algoritmo se quede atascado en opciones subóptimas, como una exploración elevada que haga que el algoritmo no explote las opciones con mejores recompensas.
- **Efecto de la distribución de recompensas.** La distribución de recompensas modifica de forma visible la facilidad de identificar el brazo óptimo. En distribución Normal, la estimación de medias se estabiliza con curvas más suaves; en Binomial aparece más variabilidad al inicio, pese a que el techo de recompensa sea alto; y en Bernoulli, el ruido inherente hace que una decisión temprana basada en pocas muestras sea especialmente arriesgada, lo que explica por qué tanto la falta de exploración como el exceso de ella se vuelven costosos.
- **Limitaciones y Trabajo Futuro:** Este estudio se ha limitado a entornos estacionarios (las probabilidades de los brazos no cambian). Una línea futura de investigación podria ser evaluar estos algoritmos en entornos *no estacionarios*, donde el brazo óptimo cambie con el tiempo, lo que obligaría a implementar técnicas de olvido o ventanas deslizantes, ya que algoritmos como UCB1 o ϵ-Greedy con $\epsilon \to 0$ fallarían al no poder readaptarse.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

El problema del bandido de k-brazos permite estudiar el equilibrio entre exploración y explotación en entornos estacionarios, pero en problemas reales las decisiones se toman dentro de un contexto o estado dinámico. Como en este tipo de escenarios suele desconocerse las recompensas que nos podemos encontrar y las mejores decisiones a tomar en cada estado, surge dentro del Aprendizaje por Refuerzo un conjunto de técnicas que consiste en tomar acciones siguiendo una política para transicionar entre estados, valorando la acción tomada según la recompensa obtenida, hasta alcanzar un nodo terminal.

El objetivo de esta práctica será desarrollar y analizar agentes basados en métodos tabulares y en métodos de control con aproximación de funciones, realizando un estudio comparativo entre ambos enfoques. Para cada uno de los enfoques, desarrollaremos los siguientes algoritmos:

1. Para los métodos tabulares, se implementarán los algoritmos de **Monte Carlo** (en sus variantes on-policy y off-policy), **SARSA** y **Q-Learning**.
2. Para el control con aproximaciones, se desarrollarán **SARSA semi-gradiente** y **Deep Q-Learning**.

Para gestionar la toma de decisiones, se implementarán las políticas de tipo ϵ-soft y ϵ-greedy, que creemos que son las más interesantes para utilizar.

Para conseguir este objetivo, vamos a utilizar Gymnasium, que es una librería que nos permite crear entornos para ensayar con los diferentes agentes. En concreto, utilizaremos el entorno de **Blackjack** que es un entorno con un espacio de acciones discreto, y el entorno **Mountain Car Continuous**, que es un entorno con un espacio de acciones continuo.

El resto de la memoria está separada en un apartado para explicar en más detalle el problema a tratar, un apartado de algoritmos en el que detallaremos más sobre los algoritmos utilizados, un apartado de evaluación/experimentos para explicar los resultados obtenidos y los experimentos realizados para obtenerlos, y por último, un apartado de conclusiones en el que resumiremos los principales resultados que hemos obtenido.

2.2. Desarrollo

En este apartado se establecen las bases teóricas necesarias para comprender el problema del aprendizaje por refuerzo en entornos complejos.

2.2.1. Marco Teórico: Aprendizaje en entornos complejos

En el capítulo anterior hemos hablado del problema del bandido de k-brazos, en el cual teniamos entornos que no cambian. En los problemas reales, las decisiones se toman dentro de un contexto o estado dinámico. Un entorno de este tipo se suele modelar como un proceso de decisión de Markov, el cual recibe una acción y devuelve un nuevo estado y una recompensa. En nuestro caso, tendremos una lista de posibles estados, con acciones disponibles para transicionar entre ellos, y al hacerlo recibiremos una recompensa. Nuestros agentes se encargarán de tomar acciones, siguiendo una política determinada, para transicionar entre estados hasta llegar a un estado terminal, y evaluando las acciones tomadas

según las recompensas obtenidas por el camino.

2.2.2. Definición Formal del Problema

Consideramos un problema en el cual tenemos un espacio de estados **S**. En cada paso de tiempo discreto $t = 1, 2, \ldots, T$, el agente selecciona una acción de las disponibles en el espacio de acciones **A** del estado actual **A(s)**. Para cada acción tomada, podemos aplicar una función de recompensa que nos devolverá la recompensa obtenida al transicionar de un estado a otro tomando una acción determinada, denotado como r(s, a, s'):

$$r(s, a, s') = \mathbb{E}[R_t | S_{t-1} = s, A_{t-1} = a, S_t = s']$$

El objetivo del agente será encontrar una política óptima determinista la cual asigne una probabilidad de 1 a la acción que maximiza el valor de estado-acción y 0 a todas las demás, denotada por $\pi_*(a|s)$:

$$\pi_*(a|s) = \begin{cases} 1 & \text{si } a = \arg\max_{a \in \mathcal{A}} q_*(s, a) \\ 0 & \text{de lo contrario} \end{cases}$$

Para que el agente pueda identificar dicha política óptima, necesitaremos utilizar las **Ecuaciones de Optimalidad de Bellman**, que descomponen el valor de una decisión en la recompensa inmediata más el valor del estado sucesor. Por un lado, tenemos la función $q_*(s, a)$ que define el valor de tomar una acción a en un estado s, siguiendo de ahí en adelante la política óptima:

$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a]$$

También tenemos la función de valor de estado $v_*(s)$, la cual nos devuelve el retorno esperado máximo que se puede obtener partiendo desde un estado s. Suponiendo que tenemos una política óptima, el valor de un estado es equivalente al valor de escoger la mejor acción posible en dicho estado:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a)$$

2.2.3. Dilema Exploración vs. Explotación

Igual que en los problemas del bandido de k-brazos, aquí volvemos a tener el dilema de exploración y explotación. Este dilema surge ya que el agente no conoce de antemano los valores de las funciones $v_*(s)$ y $q_*(s, a)$, por lo que el agente deberá de estimarlas a partir de su propia experiencia.

En este tipo de problemas, la política será la encargada de decidir las acciones que seguirá el agente, y por lo tanto elegirá cuando explorar y cuando explotar. Es importante distinguir entre dos tipos de política: la política de comportamiento, cuya función es interactuar con el entorno para recolectar datos mediante la exploración, y la política objetivo, que es la estrategia optimizada que busca explotar las acciones que maximizan la recompensa acumulada.

Mientras que la política óptima π_* es, por definición, explotadora y determinista, un agente que solo siga π_* desde el principio nunca aprendería. Si el agente no conoce el entorno, su estimación inicial de $q_*(s, a)$ será errónea, y podría quedarse atrapado en acciones subóptimas. Por lo tanto, la política de comportamiento debe ser estocástica, ya que la aleatoriedad permite que el agente explore el espacio de estados $\mathcal{S}$ y descubra recompensas que, a largo plazo, permitan al agente convergir hacia la solución óptima.

2.2.4. Trabajos Relacionados y Enfoques en la Literatura

El estudio del aprendizaje por refuerzo en entornos complejos ha evolucionado partiendo de la base de la programación dinámica formulada por Richard Bellman en su libro Dynamic Programming (1957).

- **Proximal Policy Optimization:** Algoritmo desarrollado por OpenAI para mejorar lo máximo posible una política utilizando la información disponible, pero intentando que el rendimiento no se descontrole utilizando métodos de primer orden. [9]
- **Soft Actor-Critic:** Algoritmo que optimiza una política estocástica de manera off-policy, actuando como un puente entre la optimización de políticas estocásticas y los enfoques estilo DDPG. [11]

En este trabajo nos centraremos en realizar una comparativa empírica entre los métodos tabulares, compuesto por los algoritmos de Monte Carlo, SARSA y Q-Learning, y las técnicas de control por aproximaciones, compuestas por SARSA semi-gradiente y Deep Q-Learning.

2.3. Algoritmos

En esta sección se van a discutir los diferentes algoritmos que se han implementado. Para esta práctica, se han desarrollado los algoritmos Monte Carlo, SARSA, Q-Learning, SARSA semi-gradiente y Deep Q-Learning.

2.3.1. Definición de la Estructura Base

Para garantizar una comparación justa y modular entre las diferentes estrategias, se ha implementado primero una clase abstracta base (`Agent`) que define la estructura común para todos los agentes.

Esta estructura se encarga de almacenar el entorno de Gymnasium y la tabla Q que almacenará la recompensa esperada para cada acción disponible en cada estado. También se definirán unas funciones abstractas que definirán cada agente, que son las siguientes:

1. Función para elegir la siguiente acción, según el estado actual y la política
2. Función para actualizar la tabla Q según el estado actual, la acción, la recompensa y el siguiente estado

Para las políticas también definiremos una estructura base, la cual consistirá en una clase abstracta base (`Policy`) que implementará una función para devolver una acción según el estado y otras dos para calcular las probabilidades de tomar cada acción según el estado actual.

2.3.2. Algoritmo Monte Carlo

Monte Carlo es el algoritmo más básico que vamos a implementar en esta parte. Este algoritmo creará un episodio compuesto por transiciones entre estados realizando acciones y obteniendo recompensas y aprenderá del episodio una vez terminado. Este algoritmo sigue el enfoque de Iteración Generalizada de Políticas (GPI), en el cual la política se irá evaluando para mejorarla en bucle. Este algoritmo puede ser tanto On-Policy como Off-Policy, según si la política de comportamiento y de objetivo son la misma o si son diferentes.

Para el aprendizaje, Monte Carlo a lo largo del episodio irá acumulando la experiencia obtenida al pasar por cada estado. Al acabar el episodio, se comparará la recompensa estimada y la recompensa obtenida, y se actualizará en función del error obtenido.

2.3.3. Algoritmo SARSA

SARSA es otro algoritmo que sigue el enfoque de Iteración Generalizada de Políticas (GPI) para el control y evaluación de la política. A diferencia de los métodos de Monte Carlo, SARSA utiliza el aprendizaje por Diferencia Temporal (TD), lo que significa que el aprendizaje y la actualización de los valores se hace a lo largo del episodio paso a paso, en lugar de esperar a que este termine. Además, es un algoritmo On-Policy, por lo que la política de comportamiento (la que genera la experiencia) y la política objetivo (la que se está optimizando) son exactamente la misma.

Para la actualización de $Q(S_t, A_t)$, tenemos la siguiente formula:

$$\begin{aligned} Q(S_t, A_t) \leftarrow & Q(S_t, A_t) + \\ & \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \end{aligned}$$

Siendo:

- $\boldsymbol{\alpha}$: Tasa de aprendizaje (parámetro de inicialización)
- $\boldsymbol{\gamma}$: Factor de descuento (parámetro de inicialización)
- $\mathbf{S_t}$: Estado actual
- $\mathbf{A_t}$: Acción tomada
- $\mathbf{R_{t+1}}$: Recompensa recibida
- $\mathbf{S_{t+1}}$: Siguiente estado
- $\mathbf{A_{t+1}}$: Siguiente acción tomada en S_{t+1}

En la función podemos apreciar que, desde el nuevo estado S_{t+1}, necesitamos elegir la siguiente acción que tomaremos (A_{t+1}) antes de poder realizar la actualización. Si S_{t+1} fuera un estado terminal, definiríamos $Q(S_{t+1}, A_{t+1})$ como 0.

2.3.4. Algoritmo Q-Learning

Q-Learning es otro algoritmo de aprendizaje por refuerzo que irá aprendiendo a lo largo del episodio en vez de esperar a que termine. Este algoritmo es Off-Policy, por lo que tendremos que definir una política de comportamiento y otra objetivo.

Para la actualización de $Q(S_t, A_t)$, tenemos una formula basada en la Ecuación Óptima de Bellman,

con la cual Q aproxima directamente q_* independientemente de la política. La formula es la siguiente:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Siendo:

- $\boldsymbol{\alpha}$: Tasa de aprendizaje (parámetro de inicialización)
- $\boldsymbol{\gamma}$: Factor de descuento (parámetro de inicialización)
- $\mathbf{S_t}$: Estado actual
- $\mathbf{A_t}$: Acción tomada
- $\mathbf{R_{t+1}}$: Recompensa recibida
- $\mathbf{S_{t+1}}$: Siguiente estado
- $\mathbf{A_{t+1}}$: Acción con el valor Q más alto en S_{t+1} (seleccionada por la política target)

En este caso, podemos apreciar que no hace falta seleccionar la siguiente acción para hacer la actualización, nos sirve con estimar la acción más probable.

2.3.5. Algoritmo SARSA Semi-Gradiente

SARSA Semi-Gradiente es una extensión del algoritmo SARSA tabular que introduce aproximación de funciones para estimar los valores Q. A diferencia de la versión tabular, este agente no almacena una tabla de valores discretos, sino que mantiene un vector de pesos $\mathbf{w}$ que parametriza una función de aproximación $\hat{q}(S, A, \mathbf{w})$. Esto permite trabajar con espacios de estados continuos o muy grandes.

Para la actualización de los pesos $\mathbf{w}$, tenemos la siguiente fórmula:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\,[R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})]\,\nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

En la implementación realizada se utiliza una aproximación lineal, donde:

$$\hat{q}(S, A, \mathbf{w}) = \mathbf{w}^\top \boldsymbol{\phi}(S, A)$$

siendo $\boldsymbol{\phi}(S, A)$ el vector de características del par estado-acción. Para la aproximación lineal, el gradiente es simplemente $\nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w}) = \boldsymbol{\phi}(S, A)$, por lo que la actualización se simplifica a:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot \delta_t \cdot \boldsymbol{\phi}(S_t, A_t)$$

siendo $\delta_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$ el error TD.

Los parámetros involucrados son:

- $\boldsymbol{\alpha}$: Tasa de aprendizaje (parámetro de inicialización)
- $\boldsymbol{\gamma}$: Factor de descuento (parámetro de inicialización)
- $\mathbf{S_t}$: Estado actual
- $\mathbf{A_t}$: Acción tomada en S_t
- $\mathbf{R_{t+1}}$: Recompensa recibida
- $\mathbf{S_{t+1}}$: Siguiente estado
- $\mathbf{A_{t+1}}$: Siguiente acción seleccionada por la política en S_{t+1}
- $\boldsymbol{\phi}(S, A)$: Vector de características del par estado-acción
- $\mathbf{w}$: Vector de pesos de la aproximación lineal

Al igual que en SARSA tabular, el algoritmo es On-Policy: la política de comportamiento y la política objetivo son las mismas. Si S_{t+1} es un estado terminal, se define $\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) = 0$.

2.3.6. Algoritmo Deep Q-Learning

Deep Q-Learning (DQN) es una extensión del algoritmo Q-Learning que sustituye la tabla Q por una red neuronal profunda $Q(s, a; \boldsymbol{\theta})$ para aproximar la función de valor-acción. Esto permite escalar a entornos con espacios de estados de alta dimensión o continuos, donde mantener una tabla explícita es inviable.

Al igual que Q-Learning, el algoritmo es Off-Policy y aprende a lo largo del episodio. Además, introduce una **memoria de replay** (Experience Replay) de capacidad N, que almacena las transiciones observadas y permite muestrear minibatches aleatorios para el entrenamiento, rompiendo las correlaciones temporales entre experiencias consecutivas.

Para calcular los objetivos del entrenamiento, se utiliza la fórmula basada en la Ecuación Óptima de Bellman:

$$y_j = r_j + \gamma \max_{a'} Q(\phi'_j, a'; \boldsymbol{\theta})$$

donde ϕ'_j es el siguiente estado de la transición j y $\boldsymbol{\theta}$ son los pesos actuales de la red. Si la transición j es terminal, se define $y_j = r_j$.

La pérdida utilizada para actualizar los pesos de la red es el error cuadrático medio (MSE) entre las predicciones actuales y los objetivos:

$$\mathcal{L}(\boldsymbol{\theta}) = (Q(\phi_j, a_j; \boldsymbol{\theta}) - y_j)^2$$

Los parámetros involucrados son:

- $\boldsymbol{\theta}$: Pesos de la red neuronal
- $\boldsymbol{\gamma}$: Factor de descuento (parámetro de inicialización)
- $\boldsymbol{\alpha}$ (`lr`): Tasa de aprendizaje del optimizador Adam
- **N** (`buffer_size`): Capacidad máxima de la memoria de replay D
- **B** (`batch_size`): Tamaño del minibatch extraído de D
- ϕ_j: Estado actual de la transición j
- a_j: Acción tomada en la transición j
- r_j: Recompensa recibida en la transición j
- ϕ'_j: Siguiente estado de la transición j
- `update_frequency`: Frecuencia (en pasos) con la que se entrena la red

La arquitectura de la red neuronal utilizada (`QNetwork`) recibe como entrada el vector de estado y produce como salida los valores Q para cada acción posible:

$$Q : \mathbb{R}^{d_s} \to \mathbb{R}^{|\mathcal{A}|}$$

En la implementación para MountainCar, la red tiene $d_s = 2$ entradas (posición y velocidad), dos capas ocultas de 128 neuronas con activación ReLU, y $|\mathcal{A}| = 3$ salidas (una por acción).

2.4. Experimentos

2.4.1. Entorno de Blackjack

Se trata de un juego de cartas en el que el objetivo del jugador es acercarse lo máximo posible a 21 sin sobrepasarlo y superar la mano del *dealer*. Cada partida constituye un episodio completo: el jugador comienza con una mano inicial y el *dealer* muestra una carta visible y otra oculta, por lo que la toma de decisiones se realiza con información parcial. A lo largo del episodio, el agente debe elegir entre dos acciones, pedir carta o plantarse, hasta que la partida termina. La recompensa es terminal y toma valor +1, 0 o -1 según el jugador gane, empate o pierda, respectivamente. En este entorno, el estado observable está definido por la suma actual del jugador, la carta visible del *dealer* y la presencia o no de un as utilizable, lo que permite modelar el problema como una tarea episódica discreta adecuada para métodos de Monte Carlo.

2.4.1.1. Monte Carlo

En este trabajo se comparan dos variantes de Monte Carlo sobre el entorno `Blackjack-v1`: *on-policy* y *off-policy*. En ambos casos, el estado está definido por la suma del jugador, la carta visible del crupier y la presencia de un *usable ace*, mientras que las acciones posibles son *hit* y *stick*.

En la variante *on-policy*, se utiliza una única política ε-soft con $\varepsilon = 0.1$ tanto para generar experiencia como para aprender, por lo que el agente estima directamente el valor de esa política exploratoria.

En la variante *off-policy*, la política de comportamiento es ε-greedy con $\varepsilon = 0.1$, mientras que la política objetivo es completamente greedy ($\varepsilon = 0.0$). Así, el agente explora con una política casi codiciosa, pero actualiza los valores como si siguiera una política puramente explotadora. Para corregir la diferencia entre ambas se emplea muestreo de importancia ponderado, basado en el cociente $\frac{\pi(A_t|S_t)}{b(A_t|S_t)}$. Como la política objetivo es determinista, si una acción no coincide con la greedy, el peso acumulado se anula, lo que reduce el número de actualizaciones efectivas, especialmente en los estados iniciales del episodio.

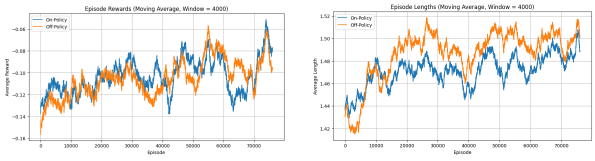
2.4.1.1.1. Bloque de entrenamiento Primeramente, se hizo una prueba previa al entrenamiento tanto en la variante *off-policy* como en la variante *on-policy*, el resultado inicial fue idéntico: una victoria, un empate y ocho derrotas. Este resultado es coherente con un agente que aún no tiene una estrategia efectiva por falta de entrenamiento. Tras esta comprobación inicial, se ejecutó el entrenamiento principal del agente. Para ambas variantes se empleó la misma configuración base, con `N_EPISODES = 80000` y `MAX_STEPS = 10`. El uso de 80000 episodios permite que el agente acumule suficiente ex-

periencia, mientras que la limitación de pasos por episodio mantiene acotada la longitud de las trayectorias y resulta razonable en un entorno donde las partidas suelen resolverse en pocos turnos. Durante este proceso, el entrenamiento registra tanto el retorno por episodio como la duración de cada partida.

En la variante **off-policy**, el agente obtiene un resultado de seis victorias y cuatro derrotas, con un porcentaje de victorias del 60 %, mientras que en la variante **on-policy**, la misma secuencia experimental permite observar una mejora también apreciable, aunque ligeramente más moderada. Después del entrenamiento el agente alcanza un porcentaje del 50 % de ratio de victorias con cinco victorias y las mismas derrotas, poniendo de manifiesto el contraste directo del efecto del aprendizaje.

2.4.1.1.2. Curvas de recompensa media y longitud de episodio Una vez descrito el bloque de entrenamiento, el primer indicador a considerar es la evolución de la recompensa a lo largo de los episodios, ya que permite comprobar si el agente mejora efectivamente con la experiencia acumulada.

2.4.1.2. Curvas de recompensa media y longitud media de episodio.



(a) Recompensa media por episodio.

(b) Duración media de episodio.

Figura 2.1: Comparativa de rendimiento de Monte Carlo en recompensa media por episodio y duración media de episodio.

En la figura 2.1 se muestra la evolución de la recompensa media y de la longitud media de los episodios. En la variante *on-policy*, la recompensa parte de valores negativos y mejora de forma progresiva, aunque con oscilaciones apreciables, algo esperable en Monte Carlo por la alta varianza asociada al uso de retornos completos. Además, la duración media de los episodios aumenta con el entrenamiento, lo que sugiere una toma de decisiones menos precipitada. En la variante *off-policy*, la recompensa también comienza en valores negativos, pero su evolución es más estable y presenta menos oscilaciones. La longitud media de los episodios, igualmente, resulta algo mayor y más regular, lo que indica un comportamiento más consistente durante el aprendizaje.

2.4.1.2.1. Mapa de la política derivada Para entender qué ha aprendido el agente es necesario examinar también la política derivada, que muestra de forma explícita qué decisión se toma en cada región del espacio de estados.

En el notebook podemos ver la gráfica que muestra en el eje horizontal la suma actual del jugador y en el eje vertical la carta visible del crupier, dos variables que determinan la decisión del agente. En ambas variantes, la política aprendida sigue la lógica esperable en Blackjack: pedir carta con sumas bajas o intermedias y plantarse cuando la suma es alta.

En la variante *on-policy*, la estrategia es más definida y conservadora. Cuando existe un *usable ace*, el agente mantiene mayor margen para pedir carta; en ausencia de este, tiende a plantarse antes, al disminuir la seguridad frente a un posible *bust*. En la variante *off-policy*, la estructura general es similar, pero las fronteras de decisión son algo menos rígidas y aparecen transiciones más flexibles entre `HIT` y `STICK`.

2.4.1.2.2. Función de valor Complementariamente, la función de valor de estado permite interpretar no solo qué decisión se toma, sino también cómo valora el agente cada situación antes de actuar.

En el notebook podemos ver las gráficas que representan la función de valor de estado $V(s)$ derivada de la tabla $Q(s,a)$, calculada en este caso como $V(s) = \text{máx}_a\, Q(s,a)$. Los ejes horizontal y vertical recogen la suma del jugador y la carta visible del crupier, mientras que la altura de la superficie indica el valor estimado del estado: los valores altos corresponden a situaciones favorables y los bajos a estados más comprometidos.

Los valores más altos se concentran en sumas elevadas del jugador, especialmente entre 19 y 21, mientras que los más bajos aparecen en sumas intermedias, donde el riesgo es mayor. Además, cuando existe un *usable ace*, la superficie suele presentar más irregularidades, no porque esos estados sean peores, sino porque aparecen con menor frecuencia y sus estimaciones acumulan más varianza. Tanto en *on-policy* como en *off-policy*, ambas superficies muestran una estructura global similar, lo que indica que en los dos casos se aprende una estrategia razonable. No obstante, la variante *off-policy* puede conservar algo más de irregularidad en ciertos estados, especialmente en los menos frecuentes, debido a la mayor varianza introducida por el muestreo de importancia ponderado.

2.4.1.2.3. Análisis de los valores de $Q(s, a)$ Finalmente, el análisis de los valores $Q(s, a)$ permite comparar directamente la conveniencia estimada de cada acción en estados concretos del juego.

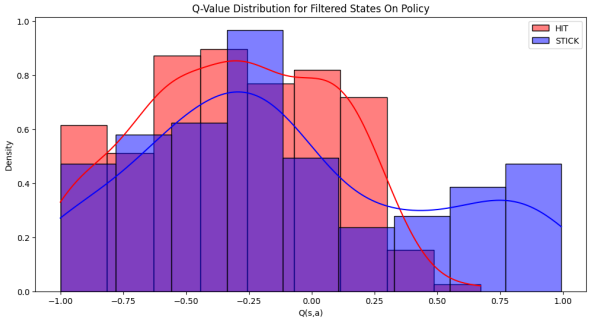


Figura 2.2: Dsitribución de valores $Q(s, a)$ en la variante On-Policy.

La figura 2.2 muestra la distribución de los valores $Q(s, a)$ para las acciones `HIT` y `STICK` en estados con suma del jugador entre 11 y 21. La acción `HIT` se concentra principalmente en valores negativos, lo que refleja que pedir carta suele ser desfavorable en muchos de estos estados, especialmente cuando aumenta el riesgo de superar 21. En cambio, `STICK` presenta una distribución más dispersa, con valores tanto negativos como positivos, ya que puede ser una decisión muy favorable en manos fuertes, pero insuficiente en situaciones débiles frente a un crupier alto. El solapamiento entre ambas distribuciones en la zona negativa indica que existen estados intrínsecamente desfavorables en los que ninguna acción ofrece una ventaja clara. En conjunto, la gráfica muestra que el agente *on-policy* distingue de forma razonable entre acciones más seguras y más arriesgadas.

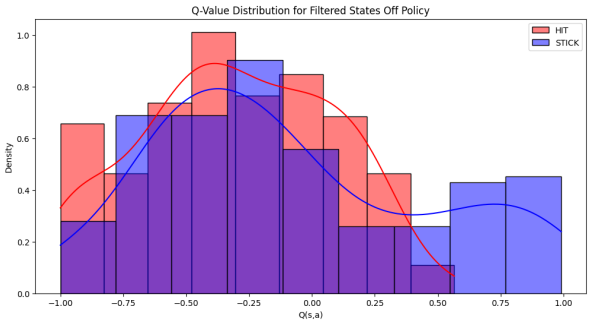


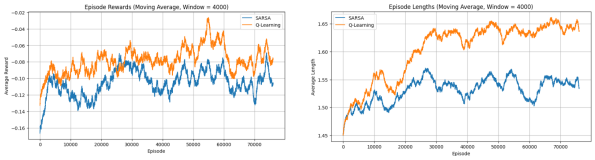
Figura 2.3: Dsitribución de valores $Q(s, a)$ en la variante On-Policy.

Por otro lado, la figura 2.3 representa la distribución de los valores $Q(s, a)$ para `HIT` y `STICK` en la variante *off-policy*. Al igual que en el caso anterior, `HIT` se concentra sobre todo en valores negativos, mientras que `STICK` presenta una distribución más extendida y alcanza con mayor claridad zonas de valor positivo, especialmente en estados favorables para plantarse. También se observa solapamiento en la región negativa, lo que indica la existencia de estados intrínsecamente desfavorables en los que ambas acciones ofrecen retornos bajos. En conjunto, la gráfica muestra que el agente *off-policy* distingue adecuadamente entre acciones convenientes y desfavorables, aunque sus estimaciones presentan algo más de irregularidad debido a la varianza introducida por el muestreo de importancia ponderado.

2.4.1.3. Diferencias Temporales (SARSA y Q-Learning)

2.4.1.3.1. Análisis de recompensas y duraciones medias por episodio En la figura 2.4 se pueden ver las gráficas que comparan SARSA y Q-Learning en cuanto a recompensa media por episodio y duración media.

En el caso de SARSA, la recompensa media parte de valores negativos, cercanos a -0.16, reflejo de una política inicial sin entrenar cuyas estimaciones



(a) Recompensa media por episodio.

(b) Duración media de episodio.

Figura 2.4: Comparativa de rendimiento de SARSA y Q-Learning en recompensa media por episodio y duración media de episodio.

de valor se basan en trayectorias poco representativas. Durante los primeros 5000 episodios se observa una mejora rápida, estabilizándose la recompensa principalmente en un rango de -0.14 a -0.07 (centrado entre -0.09 y -0.10). Paralelamente, la duración media de los episodios aumenta de forma leve, pasando de 1.45 a 1.54 pasos. Esto sugiere que el agente aprende a evitar decisiones que terminen el episodio de forma prematuramente perjudicial (como pasarse de 21), convergiendo hacia una política más estable. Este comportamiento es propio de su naturaleza *on-policy*, ya que actualiza sus valores teniendo en cuenta la política que realmente ejecuta, incluyendo las acciones exploratorias, lo que lo vuelve más conservador.

Por su parte, Q-Learning muestra un inicio ligeramente mejor (entre -0.13 y -0.14) y alcanza rápidamente recompensas medias sistemáticamente superiores, estabilizándose entre -0.08 y -0.05. Los picos de recompensa son también más altos, lo que indica el descubrimiento de trayectorias con retornos más favorables. La duración media de los episodios es notablemente mayor, alcanzando los 1.65 pasos frente a los 1.54 de SARSA. Esto denota una política más agresiva que tiende a prolongar el juego, pidiendo más cartas para buscar resultados óptimos.

Este patrón es plenamente coherente con la naturaleza *off-policy* del algoritmo, el cual actualiza los valores asumiendo siempre la mejor acción futura posible mediante el término $\max_a Q(s, a)$.

En consonancia con las diferencias teóricas estudiadas, Q-Learning logra recompensas más altas y episodios más largos al buscar la política óptima sin verse lastrado por su propia exploración. En contraste, SARSA presenta una aproximación más prudente, asumiendo los riesgos inherentes a su propia política de comportamiento durante el aprendizaje.

2.4.1.3.2. Mapa de la política derivada El estudio de la política derivada muestra de forma explícita qué decisión se toma en cada región del espacio de estados (gráfica en notebook).

Con **As utilizable**, SARSA adopta una estrategia conservadora, plantándose consistentemente desde 18; Q-Learning, en cambio, extiende la decisión de pedir carta hasta alcanzar 19, priorizando la recompensa máxima. Con **As no utilizable**, el riesgo de exceder 21 obliga a ambos agentes a ser más prudentes, plantándose desde 15. No obstante, frente a un crupier con cartas fuertes (7–10), Q-Learning mantiene la decisión de pedir carta en más estados intermedios, buscando mejorar una puntuación que de otro modo resultaría insuficiente. En definitiva, cuando existe margen de seguridad, cada algoritmo manifiesta su naturaleza: Q-Learning asume más riesgo en busca del óptimo, mientras que SARSA consolida una política más estable y cautelosa.

2.4.1.3.3. Función de valor La función de valor es continuista con lo observado en las políticas anteriores (gráfica notebook). Los estados más favorables se concentran en sumas altas (19-21), donde plantarse casi garantiza el éxito, mientras que las sumas intermedias (12-16) arrojan retornos negativos por la alta incertidumbre. La presencia de un **As utilizable** suaviza notablemente la superficie: el margen de seguridad estabiliza las estimaciones, y ambos algoritmos producen resultados prácticamente idénticos en este escenario. Las irregularidades puntuales que aún aparecen se deben a la menor frecuencia de visita de ciertos estados, no a inestabilidad de la política. Sin **As utilizable**, en cambio, la superficie se vuelve más rugosa y las diferencias entre métodos se hacen visibles: en sumas bajas, Q-Learning estima valores ligeramente superiores, pues su tendencia *off-policy* a asumir riesgos resulta ventajosa cuando el peligro de exceder 21 es mínimo. SARSA, al penalizar estos estados a través de su propia exploración, consolida estimaciones más conservadoras.

2.4.1.3.4. Análisis de la distribución de valores $Q(s,a)$ En ambos algoritmos, la curva correspondiente a `HIT` (pedir carta) se encuentra fuertemente desplazada hacia la zona negativa. Esto es coherente con la dinámica del Blackjack: pedir carta con sumas altas resulta frecuentemente en pasarse (*bust*). Por el contrario, la curva de `STICK` (plantarse) presenta una mayor dispersión. Destaca con picos en el extremo positivo derecho (plantarse con 20 o 21 es casi siempre ganador), pero mantiene presencia en la zona negativa, evidenciando que plantarse con sumas débiles frente a cartas fuertes del crupier es una decisión estadísticamente perdedora a largo plazo. Las áreas de solapamiento en la zona negativa representan estados críticos con desventaja estructural para el jugador, donde ninguna acción ofrece un valor esperado positivo.

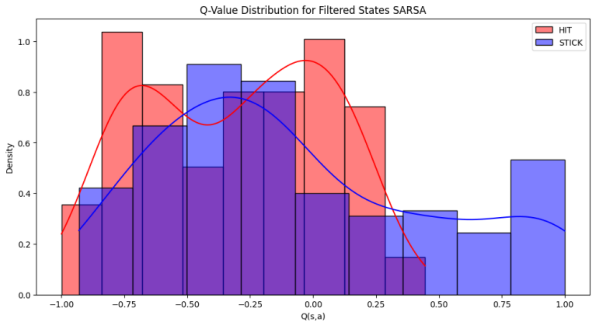


Figura 2.5: Distribución de Q-Values para SARSA.

En cuanto a SARSA, en la figura 2.5 se observa una separación relativamente clara entre ambas acciones: `HIT` se concentra marcadamente en valores negativos, mientras que `STICK` se extiende con claridad hacia el extremo positivo. Esta diferenciación refleja un aprendizaje robusto y conservador, basado en las consecuencias reales y directas de su política de exploración.

En cambio, las distribuciones de Q-Learning mostradas en la figura 2.6 aparecen más concentradas alrededor de valores cercanos a 0 (recompensas intermedias), con menor densidad en los extremos absolutos. Este fenómeno se explica por la naturaleza *off-policy* del algoritmo: al actualizar las estimaciones utilizando el operador de maximización ($\max_a Q(s',a)$), Q-Learning tiende a propagar estimaciones más optimistas hacia los estados previos. Aunque el algoritmo empuja las estimaciones hacia

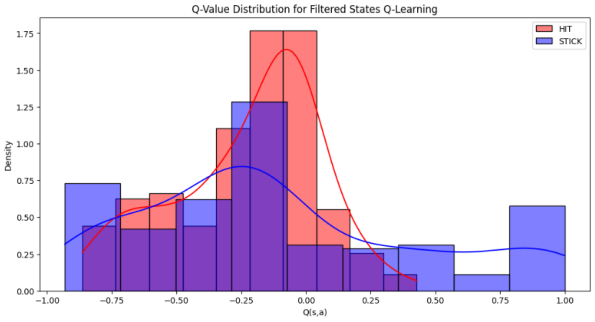


Figura 2.6: Distribución de Q-Values para Q-Learning .

valores más altos, en la práctica suaviza la distribución y agrupa los valores en la zona central, lo que refleja un sesgo optimista donde muchos de esos estados terminan resultando en recompensas moderadas.

2.4.2. Entorno de Mountain Car

Se trata de un problema de control clásico en el que el objetivo es que un coche consiga subir una pendiente pronunciada para alcanzar una meta situada en la cima de una colina, balanceandose de un lado a otro hasta conseguir la inercia suficiente. Cada episodio comienza con el coche en el fondo del valle, y termina cuando alcance el objetivo o supere un límite de 200 pasos.

Cada estado tiene la posición horizontal del coche y su velocidad actual, siendo ambas características continuas, y el agente deberá elegir entre tres acciones discretas, que serán dar marcha atras, quedarse quieto o acelerar hacia la derecha. El sistema de recompensas está diseñado para fomentar la rapidez, ya que se aplicará una penalización de -1 al agente por cada paso del episodio en el que no consiga alcanzar el objetivo.

2.4.2.1. Control con aproximaciones (SARSA semi-gradiente y Deep Q-Learning)

Para este entorno con un espacio de estados continuo, vamos a utilizar y comparar los algoritmos SARSA semi-gradiente y Deep Q-Learning, que son algoritmos capaces de funcionar en un entorno de este estilo.

En el caso de SARSA semi-gradiente, vamos a utilizar como extractor de características un *tile coding extractor*, ya que es lo suficientemente potente para este entorno y es más eficiente que otros extractores. Como política, vamos a utilizar una ϵ-greedy con un valor ϵ de 0.01, que nos ofrece la suficiente exploración para que el algoritmo converja, pero también explotará las opciones con mejor recompensa. Como parámetros del agente, vamos a utilizar un alpha de 0.05 y un gamma de 1.

En el caso de Deep Q-Learning, hemos elegido como política una ϵ-greedy con un valor ϵ de 0.01, que nos ofrece la suficiente exploración para que el algoritmo converja, pero también explotará las opciones con mejor recompensa. Para la red neuronal, vamos a utilizar un perceptrón multicapa con 2 capas ocultas de 128 neuronas, que es lo suficiente complejo para este entorno. En nuestra implementación, hemos añadido una optimización para que la red neuronal aprenda solo cada 4 acciones, mejorando el aprendizaje debido a la diversidad de los datos del Replay Buffer y reduciendo el tiempo de ejecución.

Nuestra implementación de la red neuronal la haremos con pytorch. Como indica su página su página oficial (reproducibilidad pytorch), no se asegura que los resultados obtenidos sean 100 % reproducibles. Por ello, para las explicaciones del algoritmo se tendrá en cuenta por un lado los resultados actuales de la última ejecución (en el entorno Google Colab con CPU), y por otro lado se tendrán en cuenta los diferentes resultados obtenidos en diferentes ejecuciones. De esta forma, además de analizar los últimos resultados obtenidos, podremos analizar resultados comunes entre todas las ejecuciones que son producidas por el algoritmo sin importar los resultados concretos. Repitiendo la ejecución del notebook en Google Colab con CPU los resultados han sido identicos, pero no aseguramos que en el futuro lo sigan siendo por los motivos comentados.

2.4.2.1.1. Bloque de entrenamiento Antes del entrenamiento, se hizo una prueba de 5 episodios en el entorno con ambos algoritmos, en la cual ninguno de los algoritmos pudo alcanzar la meta en ninguno de los episodios. Este resultado es coherente ya que el agente todavía no tiene una estrategia debido a la falta de entrenamiento.

Para el entrenamiento de ambos algoritmos hemos empleado la misma configuración base, con N_EPISODES = 5000 y MAX_STEPS = 200. Aunque en el anterior entorno utilizamos 80000 episodios, en este entorno hemos comprobado que 5000 es suficiente para que ambos algoritmos converjan, por lo que no vemos necesario aumentar el número de episodios. La longitud del episodio mantiene acotada su duración, y hemos comprobado que el valor por defecto del entorno (200) es un valor que funciona bastante bien, ya que da tiempo suficiente al algoritmo para llegar al objetivo, pero no permite trayectorias demasiado largas que no nos interesan.

Después del entrenamiento, hemos vuelto a ejecutar la prueba de 5 episodios, en la cual ambos algoritmos han conseguido alcanzar la meta en todos los episodios. Aunque 5 episodios es una muestra muy pequeña, nos sirve para comprobar la mejora del algoritmo después del entrenamiento. Aún así, además de si ha alcanzado el objetivo o no, también nos interesa saber cuantos pasos ha tardado el agente, que lo analizaremos con los resultados de las gráficas.

2.4.2.1.2. Número de pasos por episodio En la figura 2.7 podemos ver la gráfica que muestra la evolución de los pasos por episodio que han tardado los algoritmos SARSA semi-gradiente y Deep Q-Learning.

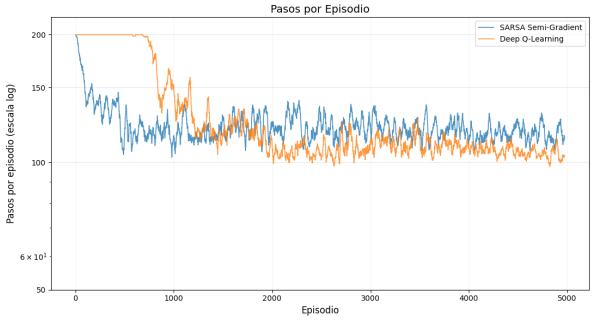


Figura 2.7: Pasos por episodio para control por aproximaciones.

En el caso de SARSA semi-gradiente, podemos ver una gran eficiencia desde las primeras etapas del entrenamiento, presentando una caida drástica en

el número de pasos en los primeros 500 episodios. Esta mejora inicial se debe a la función lineal de SARSA, ya que al usar aproximadores lineales, el algoritmo es capaz de construir una política efectiva incluso con una cantidad de datos limitada. A largo plazo vemos que el agente se estabiliza entre 110 y 130 pasos por episodio, con ciertas oscilaciones, pero teniendo aún así una tendencia bastante consistente y eficiente.

Por otro lado, Deep Q-Learning no presenta ninguna mejora los primeros 750 episodios, ya que el entrenamiento de la red neuronal y llenar el Replay Buffer de experiencias necesita más tiempo. Sin embargo, a partir de aquí podemos ver una mejora radical entre los episodios 750 y 850, llegando hasta los 130 pasos aproximadamente. A partir de aquí, el algoritmo presenta cierta varianza, pero vemos que sigue bajando la media de los pasos por episodio, llegando a converger en el episodio 2000 y manteniendose en un número de pasos entre 100 y 125.

Podemos sacar una diferencia clave entre ambos algoritmos, la cual ha sido el tiempo en converger. Para todas las ejecuciones de DQL, SARSA semi-gradiente ha conseguido una convergencia más rápida. Como hemos comentado, la red neuronal y el Replay Buffer de DQL retrasan el aprendizaje, mientras que SARSA semi-gradiente consigue un aprendizaje más continuo desde el principio hasta llegar a converger.

Por otro lado, si hablamos del número de pasos por episodio, en general ambos algoritmos han conseguido resultados muy similares en todas las ejecuciones. En este caso concreto hemos obtenido mejores resultados con DQL, pero en otras ejecuciones hemos obtenido con este mismo algoritmo resultados más inconsistentes y en general peores que los de SARSA semi-gradiente. Esto puede deberse a que SARSA es On-Policy, y además usa aproximadores lineales, por lo que puede generalizar de forma muy suave en espacios de estados continuos. DQL, al ser Off-Policy y usar redes neuronales, es un algoritmo más complejo que en problemas tan sencillos como este puede puede llegar a ser más inconsistente que SARSA. Aún así, el caracter optimista de DQL puede hacer que el algoritmo genere mejores resultados, como ha sido en este caso.

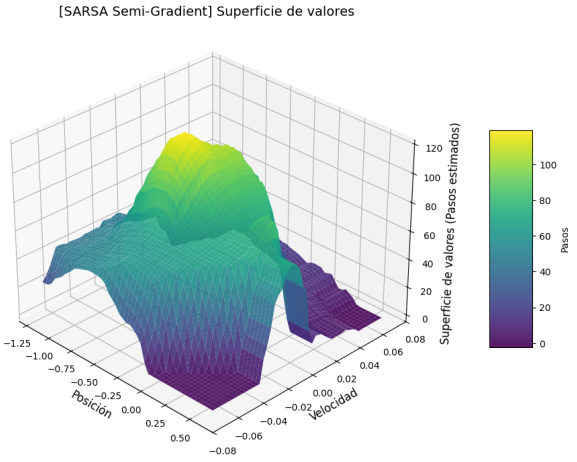


Figura 2.8: Superficie de valores de estado para SARSA semi-gradiente.

2.4.2.1.3. Superficie de valores de estado
En las figuras 2.8 y 2.9 podemos ver las gráficas de la superficie de valores de estado para SARSA semi-gradiente y Deep Q-Learning, en las cuales los agentes harán una estimación de cuantos pasos cree que le quedan para llegar al objetivo según el estado actual.

La gráfica 2.8 corresponde a SARSA semi-gradiente. Podemos ver que el rango Z llega hasta los 120 pasos, que es una estimación realista viendo la gráfica 2.7. Las arrugas de la superficie indican que el algoritmo ha podido identificar con bastante precisión los puntos donde gana o pierde la inercia necesaria para poder subir la montaña. Esto puede deberse gracias a que el algoritmo es On-Policy y sus aproximadores lineales consiguen una estimación bastante precisa de cada estado.

La gráfica 2.9 corresponde a Deep Q-Learning. En este caso el rango Z solo llega hasta los 70 pasos, y además en la base de la gráfica podemos ver valores negativos. Esto se debe a que el agente tiene un comportamiento demasiado optimista, y cree que puede terminar el episodio más rápido de lo que en realidad puede, caracter muy típico del algoritmo Q-Learning. La gráfica en general es muy suave, por lo que parece que la red neuronal no está consiguiendo distinguir entre estados similares pero con consecuencias distintas, eliminando detalles importantes de la gráfica.

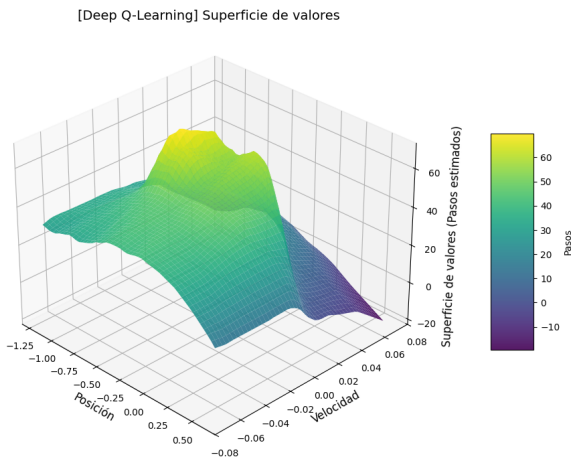


Figura 2.9: Superficie de valores de estado para Deep Q-Learning.

SARSA, al ser On-Policy y más simple, está siendo un algoritmo más realista, que aprende basándose en lo que está haciendo cada paso y consigue crear un mapa de la montaña muy preciso. Mientras tanto, DQN es demasiado optimista y además, al usar una red neuronal en un problema tan sencillo, está simplificando demasiado el terreno, cosa que puede hacer que las decisiones tomadas no sean del todo correctas y pueda llegar a tomar más tiempo del necesario en llegar al objetivo. Estos resultados han sido identicos para todas las ejecuciones de DQL.

2.4.2.1.4. Mapa de elecciones del agente

En la figura 2.10 podemos ver las decisiones que tomaría el agente según la posición y velocidad actuales, y un ejemplo de camino que tomaría para llegar a la meta. Hay que tener en cuenta que el punto de inicio del coche en el entorno es aleatorio, por lo que más que analizar cuanto tiempo toma el agente en llegar a la meta en la gráfica, debemos de analizar las areas de decisión.

Ambas gráficas muestran como el agente debe "balancearse" para llegar al objetivo, tomando la decisión de ir hacia la derecha con velocidad positiva para seguir subiendo la colina, y yendo hacia la izquierda con velocidad negativa para coger inercia. También podemos ver en ambas gráficas que, a pesar de tener un poco de ruido, ambos agentes han conseguido areas de decisión bastante suaves y bien

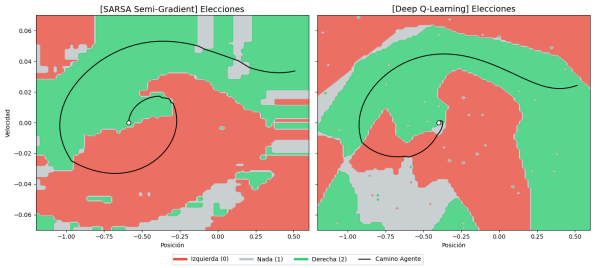


Figura 2.10: Mapa de elecciones del agente.

delimitadas, y caminos hacia el objetivo con una forma de espiral muy limpia.

Una diferencia que podemos apreciar es que SARSA semi-gradiente genera areas de decisión con forma más cuadrada, mientras que en Deep Q-Learning podemos ver formas más irregulares. SARSA genera la gráfica de esta manera debido al uso de *tile coding*, ya que divide el espacio de estados en una cuadricula. En el caso de DQL, al usar redes neuronales con funciones matemáticas continuas, no dividen el espacio de estados en bloques tan diferenciables. Esto ha sucedido en todas las ejecuciones de DQL.

Aunque ambos algoritmos consiguen converger a una buena política y consiguen gráficas con regiones de decisión muy suaves y bien delimitadas, con el número de pasos por episodio mostrado en 2.7 podemos ver cual de los dos algoritmos ha conseguido una política que se adapte de mejor manera al problema. En este caso concreto, podemos observar que DQL ha conseguido adaptarse de mejor manera al problema y generar unos mejores resultados.

2.5. Conclusiones

El estudio realizado sobre el aprendizaje en entornos complejos, nos ha permitido corroborar empíricamente los dilemas fundamentales del aprendizaje por refuerzo en entornos más realistas que los estudiados en el problema del bandido de k-brazos. A continuación, se resumen las principales conclusiones:

- **Comparación entre Monte Carlo On-Policy y Off-Policy.** Con el algoritmo Monte Carlo, hemos podido comprobar la diferencia entre utilizar la misma política de comporta-

miento y objetivo frente a emplear dos políticas distintas. La variante on-policy, al aprender directamente de su propia política exploratoria ($\epsilon - soft$ con $\epsilon = 0.1$), mostró una mejora progresiva aunque con mayor variabilidad en las curvas de recompensa, alcanzando un ratio de victorias del 50 % tras el entrenamiento. Por su parte, la variante off-policy, gracias al uso del muestreo de importancia ponderado, logró un comportamiento más estable y una mejora más pronunciada, alcanzando un 60 % de victorias. Sin embargo, esta mayor estabilidad tiene un coste: el muestreo de importancia ponderado introduce varianza adicional en los estados menos visitados, lo que se refleja en ciertas irregularidades en la función de valor y en la política derivada. Ambas variantes aprendieron estrategias razonables y coherentes con la dinámica del Blackjack, confirmando que Monte Carlo es un método válido para entornos episódicos discretos, aunque su alta varianza inherente exige un número elevado de episodios para converger de forma fiable.

- **Comparación entre SARSA y Q-Learning.** Después de Monte Carlo, hemos continuado haciendo un análisis comparativo entre SARSA y Q-Learning. Al igual que en el problema del acantilado, hemos podido ver el comportamiento conservador de SARSA frente al arriesgado de Q-Learning. En este caso, al no tener una penalización tan alta por caida, la naturaleza Off-Policy de Q-Learning, que le permite explorar opciones nuevas a la vez que explota la opción óptima, ha conseguido generar mejores resultados que SARSA.
- **Comparación entre SARSA semi-gradiente y Deep Q-Learning.** Con SARSA semi-gradiente y Deep Q-Learning hemos podido probar los algoritmos de aprendizaje por refuerzo en entornos con estados continuos, como ha sido en nuestro caso el de *Mountain Car*. Aunque ambos algoritmos han funcionado bastante bien, hemos podido comprobar como un algoritmo como SARSA semi-gradiente, que es On-Policy y utiliza aproximadores lineales, puede adaptarse mejor a problemas sencillos que otros algoritmos más complejos como DQL, que es Off-Policy y utiliza una red neuronal. Esto lo demuestra que a lo largo de varias ejecuciones de DQL hemos observado resultados más inconsistentes y en varias ocasiones peores que los de SARSA. Sin embargo, el caracter optimista de DQL puede llegar a generar mejores resultados, como ha sido el caso de nuestra última ejecución.

Bibliografía

[1] Sahana Ramnath. (2018). *MultiArmedBandit_RL*. GitHub. https://github.com/SahanaRamnath/MultiArmedBandit_RL

[2] Afshin Khadangi. (2020). *Multi-armed-Bandits*. GitHub. https://github.com/akhadangi/Multi-armed-Bandits

[3] John Myles White. (2016). *BanditsBook*. GitHub. https://github.com/johnmyleswhite/BanditsBook/

[4] Sutton, R. S., y Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2.ª ed.). MIT Press. http://incompleteideas.net/book/the-book-2nd.html

[5] Vermorel, J., y Mohri, M. (2005). *Multi-armed bandit algorithms and empirical evaluation*. European Conference on Machine Learning. https://link.springer.com/chapter/10.1007/11564096_42

[6] Auer, P., Cesa-Bianchi, N., y Fischer, P. (2002). *Finite-time analysis of the multiarmed bandit problem*. Machine Learning, 47(2), 235–256.

[7] Kuleshov, V., y Precup, D. (2014). *Algorithms for multi-armed bandit problems*. Journal of Machine Learning Research. https://www.cs.mcgill.ca/~vkules/bandits.pdf

[8] Luce, R. D. (1959). *Individual Choice Behavior: A Theoretical Analysis*. John Wiley and Sons. https://link.springer.com/article/10.1023/A:1013689704352

[9] OpenAI (2017). *Proximal Policy Optimization*. https://spinningup.openai.com/en/latest/algorithms/ppo.html

[10] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529-533. https://doi.org/10.1038/nature14236

[11] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, Sergey Levine Soft Actor-Critic Algorithms and Applications https://arxiv.org/abs/1812.05905

[12] Uso de Gemini y otros LLM para ayuda en la generación de código y redacción de la memoria.